

Базы данных
Лабораторная работа 7

Романенко Мария Б23-513

Оптимизация и анализ выполнения запросов

Цель

Изучить влияние индексов и структуры запросов на производительность.

Содержание работы

- снять планы выполнения **EXPLAIN (ANALYZE, BUFFERS)** до индексов;
- спроектировать индексы:
 - составной,
 - частичный,
 - покрывающий;
- повторно снять планы;
- сравнить результаты.

Запрос 1: Поиск долгосрочной аренды больших квартир

Цель: Продемонстрировать работу **составного** и **частичного** индексов.

Логика: Найти адреса квартир площадью от 40 кв.м., которые сданы на долгий срок (от 12 месяцев), ФИО риелтора, сопровождающего сделку.

```
SELECT p.Address, p.Area, e.FullName, rc.MonthlyRent
FROM RentContract rc
JOIN Property p ON rc.PropertyId = p.PropertyId
JOIN Employee e ON rc.RealtorId = e.EmployeeId
WHERE p.Type = 'квартира' AND p.Area >= 40.0
      AND rc.DurationMonths >= 12;
```

До индексации

	QUERY PLAN	
	text	🔒
1	Merge Join (cost=57.38..59.87 rows=92 width=119) (actual time=0.815..0.853 rows=39.00 loops=1)	
2	Merge Cond: (e.employeeid = rc.realtorid)	
3	Buffers: shared hit=250	
4	-> Index Scan using employee_pkey on employee e (cost=0.28..44.01 rows=649 width=52) (actual time=0.014..0.051 rows=92.00 loops=1)	
5	Index Searches: 1	
6	Buffers: shared hit=182	
7	-> Sort (cost=56.26..56.49 rows=92 width=75) (actual time=0.783..0.785 rows=39.00 loops=1)	
8	Sort Key: rc.realtorid	
9	Sort Method: quicksort Memory: 28kB	
10	Buffers: shared hit=68	
11	-> Merge Join (cost=47.11..53.26 rows=92 width=75) (actual time=0.151..0.769 rows=39.00 loops=1)	
12	Merge Cond: (p.propertyid = rc.propertyid)	
13	Buffers: shared hit=68	
14	-> Index Scan using property_pkey on property p (cost=0.28..159.28 rows=549 width=67) (actual time=0.011..0.593 rows=512.00 loo...	
15	Filter: ((area >= 40.0) AND ((type)::text = 'квартира'::text))	
16	Rows Removed by Filter: 2488	
17	Index Searches: 1	
18	Buffers: shared hit=62	
19	-> Sort (cost=46.83..48.13 rows=520 width=16) (actual time=0.115..0.124 rows=252.00 loops=1)	
20	Sort Key: rc.propertyid	
21	Sort Method: quicksort Memory: 34kB	
22	Buffers: shared hit=6	
23	-> Seq Scan on rentcontract rc (cost=0.00..23.38 rows=520 width=16) (actual time=0.013..0.065 rows=252.00 loops=1)	
24	Filter: (durationmonths >= 12)	
25	Rows Removed by Filter: 248	
26	Buffers: shared hit=6	
27	Planning:	
28	Buffers: shared hit=104	
29	Planning Time: 1.951 ms	
30	Execution Time: 0.906 ms	

-- Composite Index

-- поиск недвижимости по типу и площади одновременно

```
CREATE INDEX idx_prop_type_area ON Property(Type, Area);
```

-- Partial Index

-- Индексирует только долгосрочные договоры

```
CREATE INDEX idx_rent_longterm ON RentContract(PropertyId,  
RealtorId)
```

```
WHERE DurationMonths >= 12;
```

После

	QUERY PLAN	
	text	
1	Merge Join (cost=102.55..111.05 rows=43 width=119) (actual time=0.384..0.407 rows=39.00 loops=1)	
2	Merge Cond: (e.employeeid = rc.realtorid)	
3	Buffers: shared hit=64 read=5	
4	-> Index Scan using employee_pkey on employee e (cost=0.27..38.82 rows=300 width=52) (actual time=0.009..0.022 rows=92.00 loops=...	
5	Index Searches: 1	
6	Buffers: shared hit=8	
7	-> Sort (cost=98.21..98.32 rows=43 width=75) (actual time=0.365..0.367 rows=39.00 loops=1)	
8	Sort Key: rc.realtorid	
9	Sort Method: quicksort Memory: 28kB	
10	Buffers: shared hit=56 read=5	
11	-> Hash Join (cost=36.96..97.04 rows=43 width=75) (actual time=0.227..0.352 rows=39.00 loops=1)	
12	Hash Cond: (p.propertyid = rc.propertyid)	
13	Buffers: shared hit=56 read=5	
14	-> Bitmap Heap Scan on property p (cost=21.56..79.28 rows=515 width=67) (actual time=0.123..0.219 rows=512.00 loops=1)	
15	Recheck Cond: (((type)::text = 'квартира'::text) AND (area >= 40.0))	
16	Heap Blocks: exact=50	
17	Buffers: shared hit=50 read=5	
18	-> Bitmap Index Scan on idx_prop_type_area (cost=0.00..21.43 rows=515 width=0) (actual time=0.112..0.112 rows=512.00 loops=1)	
19	Index Cond: (((type)::text = 'квартира'::text) AND (area >= 40.0))	
20	Index Searches: 1	
21	Buffers: shared read=5	
22	-> Hash (cost=12.25..12.25 rows=252 width=16) (actual time=0.093..0.093 rows=252.00 loops=1)	
23	Buckets: 1024 Batches: 1 Memory Usage: 21kB	
24	Buffers: shared hit=6	
25	-> Seq Scan on rentcontract rc (cost=0.00..12.25 rows=252 width=16) (actual time=0.011..0.062 rows=252.00 loops=1)	
26	Filter: (durationmonths >= 12)	
27	Rows Removed by Filter: 248	
28	Buffers: shared hit=6	
29	Planning:	
30	Buffers: shared hit=145 read=2	
31	Planning Time: 2.885 ms	
32	Execution Time: 0.441 ms	

Запрос 2: Отчет по выручке риелторов с продаж за наличные

Цель: Продемонстрировать работу покрывающего индекса.

Логика: Посчитать суммарную стоимость сделок, закрытых каждым риелтором при условии оплаты наличными.

```
SELECT sc.RealtorId, e.FullName, SUM(sc.TotalPrice) as TotalRevenue
FROM SaleContract sc
JOIN Employee e ON sc.RealtorId = e.EmployeeId
WHERE sc.PaymentType = 'наличные'
GROUP BY sc.RealtorId, e.FullName
ORDER BY TotalRevenue DESC;
```

До индексации

	QUERY PLAN text	
1	Sort (cost=143.52..145.61 rows=836 width=84) (actual time=0.785..0.788 rows=60.00 loops=1)	
2	Sort Key: (sum(sc.totalprice)) DESC	
3	Sort Method: quicksort Memory: 29kB	
4	Buffers: shared hit=46	
5	-> HashAggregate (cost=92.50..102.95 rows=836 width=84) (actual time=0.752..0.766 rows=60.00 loops=1)	
6	Group Key: sc.realtorid, e.fullname	
7	Batches: 1 Memory Usage: 73kB	
8	Buffers: shared hit=46	
9	-> Hash Join (cost=17.75..86.23 rows=836 width=60) (actual time=0.176..0.553 rows=836.00 loops=1)	
10	Hash Cond: (sc.realtorid = e.employeeid)	
11	Buffers: shared hit=46	
12	-> Seq Scan on salecontract sc (cost=0.00..66.25 rows=836 width=12) (actual time=0.031..0.293 rows=836.00 loo...	
13	Filter: ((paymenttype)::text = 'наличные':text)	
14	Rows Removed by Filter: 1664	
15	Buffers: shared hit=35	
16	-> Hash (cost=14.00..14.00 rows=300 width=52) (actual time=0.130..0.130 rows=300.00 loops=1)	
17	Buckets: 1024 Batches: 1 Memory Usage: 34kB	
18	Buffers: shared hit=11	
19	-> Seq Scan on employee e (cost=0.00..14.00 rows=300 width=52) (actual time=0.010..0.059 rows=300.00 loo...	
20	Buffers: shared hit=11	
21	Planning Time: 0.295 ms	
22	Execution Time: 0.873 ms	

-- Покрывающий индекс

```
CREATE INDEX idx_sale_payment_cover ON SaleContract(PaymentType,
RealtorId) INCLUDE (TotalPrice);
```

После

	QUERY PLAN text	
1	Sort (cost=124.18..126.27 rows=836 width=84) (actual time=0.706..0.709 rows=60.00 loops=1)	
2	Sort Key: (sum(sc.totalprice)) DESC	
3	Sort Method: quicksort Memory: 29kB	
4	Buffers: shared hit=12 read=8	
5	-> HashAggregate (cost=73.16..83.61 rows=836 width=84) (actual time=0.675..0.689 rows=60.00 loops=1)	
6	Group Key: sc.realtorid, e.fullname	
7	Batches: 1 Memory Usage: 73kB	
8	Buffers: shared hit=12 read=8	
9	-> Hash Join (cost=18.03..66.89 rows=836 width=60) (actual time=0.174..0.486 rows=836.00 loops=1)	
10	Hash Cond: (sc.realtorid = e.employeeid)	
11	Buffers: shared hit=12 read=8	
12	-> Index Only Scan using idx_sale_payment_cover on salecontract sc (cost=0.28..46.91 rows=836 width=12) (actual time=0.101..0.285 rows=836.00 lo...	
13	Index Cond: (paymenttype = 'наличные':text)	
14	Heap Fetches: 0	
15	Index Searches: 1	
16	Buffers: shared hit=1 read=8	
17	-> Hash (cost=14.00..14.00 rows=300 width=52) (actual time=0.068..0.069 rows=300.00 loops=1)	
18	Buckets: 1024 Batches: 1 Memory Usage: 34kB	
19	Buffers: shared hit=11	
20	-> Seq Scan on employee e (cost=0.00..14.00 rows=300 width=52) (actual time=0.009..0.038 rows=300.00 loops=1)	
21	Buffers: shared hit=11	
22	Planning:	
23	Buffers: shared hit=35 read=1	
24	Planning Time: 1.245 ms	
25	Execution Time: 0.746 ms	

Таблица сравнения показателей

Запрос	Показатель	До индексации	После индексации
1	соединения	Merge Join	Hash Join
	buffers read	104	2
	exec. Time	0.906	0.441
2	соединения	Hash Join	Hash Join
	buffers read	46	1
	exec. Time	0.873	0.746